



An AI-powered ML consultant platform
For final-year engineering projects

Complete Development Roadmap & Team Work Distribution

Generated: July 16, 2026

1. Architecture Stack Overview

BuildMyML uses a simplified, production-inspired architecture designed for a final-year engineering project. The stack prioritizes clarity, modularity, and explainability over enterprise complexity.

Core Stack:

Layer	Technology	Purpose
Frontend	React (Vite)	Chat UI, Clarifications, Download
Backend API	FastAPI	Auth, Sessions, State, Code Generation
Orchestration	n8n	Sequential workflow, Waits, Retries
AI Service	FastAPI	Agents, RAG, LLM Client
Database	PostgreSQL + pgvector	State, Embeddings, RAG
File Storage	Local Filesystem	Generated Project ZIPs
LLM	External API (OpenAI/Anthropic)	All AI completions

Flow: Frontend → Backend (public API) → n8n (orchestrator) → AI Service (agents + RAG) → PostgreSQL

2. Simplified Design Decisions

The following enterprise components were removed to match project scope (8–10 weeks, small team). Each removal trades off a feature for reduced complexity:

Component	Decision	Rationale
Redis	■ Removed	Session state fits in PostgreSQL; 2–3s polling suffices for demo
Microservices	■ Removed	One AI Service with modules is clearer; no orchestration ops burden
S3 / Object Storage	■ Removed	ZIPs are small & low-volume; local storage sufficient
WebSockets / SSE	■ Removed (v1)	HTTP polling is simpler; upgrade in v2 if needed
Message Bus	■ Removed	n8n already orchestrates; a bus duplicates that role
Graph RAG	■ Removed (v1)	Dense retrieval + metadata filters in pgvector work well
Event Sourcing	■ Removed	Versioned JSON state + optional step_logs table suffices

Result: A system that is *production-inspired* (API contracts, structured logging, modular design) without microservice ops pain. Easy to explain in a viva (one diagram, one database, one orchestrator).

3. System Topology & Component Diagram

The architecture follows a clear separation of concerns. Each deployment unit has a single owner and responsibility:

Four Deployment Units:

Unit	Owner	Technology	Role
Frontend	Teammate (Aayush)	React + Vite	Chat UI, Clarifications, Download
Backend API	Teammate (Aayush)	FastAPI	Public endpoints, Auth, Sessions, State CRUD
AI Service	Hima + Harshita	FastAPI	Agents, RAG, LLM client, all AI log c
n8n	Hima	n8n (Docker)	Orchestration: order, waits, retries
PostgreSQL	Shared	PostgreSQL + pgvector	Durable state, embeddings, workflow history

Architecture Diagram (Text):

```

■ FRONTEND (Aayush) ■
■ React – Chat UI, Clarifications, Download ■
■ HTTPS / REST (public API only)
■ BACKEND API – FastAPI (Aayush) ■
■ Auth · Sessions · Workflow Client · State Module · Code Gen ■
■ Webhooks ■
■ (trigger / resume) ■
■ n8n – Orchestrator (Hima) ■
■ Sequential pipeline · Clarification wait · Retry loops ■
■ HTTP (internal) ■
■ AI SERVICE – FastAPI (Hima + Harshita) ■
■ LLM Client ■ ■ RAG ■ ■ Agents (requirement, ■ ■
■ (shared) ■ ■ Module ■ ■ clarification, dataset, ■ ■
■ ■ ■ ■ ■ model, metrics, experiment, ■ ■
■ ■ ■ ■ ■ rai, critic, blueprint) ■ ■
■ PostgreSQL + pgvector ■
■ users · sessions · workflow states · step logs · embeddings ■

```



4. AI Service Internal Structure

The AI Service is organized as a modular monolith: one FastAPI app with independent agent packages and shared utilities.

Folder Structure:

```
ai_service/
  ■■■ main.py # FastAPI app – routes only
  ■■■ llm/
  ■ ■■■ __init__.py
  ■ ■■■ client.py # Shared LLM client (all agents use this)
  ■ ■■■ config.py # Model selection, timeouts, retries
  ■■■ rag/
  ■ ■■■ __init__.py
  ■ ■■■ retriever.py # Query + similarity search
  ■ ■■■ ingest.py # Offline: chunk → embed → pgvector
  ■ ■■■ embeddings.py # Sentence Transformers wrapper
  ■■■ agents/
  ■■■ requirement/
  ■ ■■■ __init__.py
  ■ ■■■ agent.py # Stateless module
  ■ ■■■ prompts.py # Requirement extraction prompt
  ■■■ clarification/
  ■ ■■■ agent.py
  ■ ■■■ prompts.py
  ■■■ dataset/
  ■■■ model/
  ■■■ metrics/
  ■■■ experiment/
  ■■■ responsible_ai/
  ■■■ critic/
  ■■■ blueprint/
```

Key Principle: Agents are *stateless*. n8n passes `state_slice` → agent processes → agent returns structured JSON → n8n merges into state. No agent imports or calls another agent.

5. Module Ownership & Responsibilities

Module / Component	Owner	Responsibility
System Architecture	Hima	Design, decisions, final integration
RAG Pipeline	Hima + Harshita	Document ingest, retrieval, pgvector
Requirement Agent	Hima	Extract problem, domain, objectives
Clarification Agent	Hima	Ask & merge follow-up answers
RAG Agent	Hima	Retrieve & integrate documentation
Critic Agent	Hima	Validate recommendations, retry logic
Blueprint Agent	Hima	Generate final project blueprint
FastAPI Backend	Aayush	Public API, auth, sessions, download
Dataset Agent	Harshita	Discover & score datasets
Model Agent	Harshita	Recommend models with explanations
Metrics Agent	Harshita	Suggest evaluation metrics
Experiment Agent	Harshita	Plan baselines, CV, tuning
Responsible AI Agent	Harshita	Flag bias, fairness, privacy concerns
React Frontend	Aayush	Chat UI, Blueprint viewer, Downloads
n8n Workflows	Hima	Orchestration, sequencing, waits
Code Generator	Aayush	Blueprint JSON → starter project ZIP
Documentation	Rishitha	SRS, SDS, API docs, Architecture diagrams

6. End-to-End Data Flow

When a user submits an ML idea, the system orchestrates a multi-step pipeline. State is the single source of truth, stored in PostgreSQL and read/written via the Backend API.

Pipeline Sequence:

1. User submits ML idea → POST /api/v1/sessions
Backend creates session & WorkflowState in PostgreSQL
2. Backend triggers n8n → POST /webhook/start
3. n8n loads state → GET /internal/state/{session_id}
4. n8n calls Requirement Agent → POST /agents/requirement/run
Agent extracts: problem, domain, constraints, objectives
5. n8n merges output → PATCH /internal/state/{session_id}
6. n8n calls Clarification Agent → POST /agents/clarification/run
Agent asks: dataset available? binary or multi-class? real-time?
7. n8n pauses workflow → sets status: awaiting_clarification
8. Frontend polls → GET /api/v1/sessions/{id}
Shows questions to user
9. User submits answers → POST /api/v1/sessions/{id}/clarifications
Backend triggers n8n resume → POST /webhook/resume
10. n8n merges clarification answers into state
11. n8n calls RAG Agent → POST /rag/retrieve
Retrieves relevant documentation chunks
12. n8n continues with agents in sequence:
Dataset Agent → Model Agent → Metrics Agent →
Experiment Agent → Responsible AI Agent → Critic Agent
13. Critic validates all recommendations
If failed: retry agent (max 2 attempts)
If passed: continue
14. n8n calls Blueprint Agent → generates final blueprint JSON
15. n8n calls Code Generator (Backend) → POST /internal/codegen/generate
Converts blueprint → starter project ZIP
16. n8n marks workflow complete
17. Frontend → GET /api/v1/sessions/{id}/download
User downloads generated project

7. WorkflowState – Single Source of Truth

All workflow data is stored as a single JSON document in PostgreSQL. This eliminates state inconsistencies and makes debugging straightforward.

WorkflowState Structure:

```
{
  "session_id": "uuid-1234",
  "workflow_run_id": "uuid-5678",
  "created_at": "2024-01-15T10:00:00Z",
  "status": "running", # running | awaiting_clarification | completed | failed
  "current_step": "model_recommendation",

  "input": {
    "raw_idea": "I want to predict customer churn",
    "clarification_answers": [
      { "question": "Is this binary or multi-class?", "answer": "binary" },
      { "question": "Real-time prediction needed?", "answer": "no" }
    ]
  },

  "requirements": {
    "problem": "Customer churn prediction",
    "domain": "SaaS / Telecom",
    "task_type": "classification",
    "constraints": "interpretability required",
    "objectives": ["maximize f1", "minimize false negatives"]
  },

  "rag_context": {
    "chunks": [
      { "doc_id": "sklearn-rf", "content": "...", "score": 0.92 },
      { "doc_id": "xgboost-guide", "content": "...", "score": 0.88 }
    ],
    "citations": ["Scikit-learn Docs", "XGBoost Paper"]
  },

  "recommendations": {
    "datasets": [
      {
        "name": "Telco Customer Churn (Kaggle)",
        "link": "kaggle.com/...",
        "size": "7043 rows",
        "score": 0.94
      }
    ],
    "models": [
      { "name": "XGBoost", "confidence": 0.92, "reason": "..." },
      { "name": "Random Forest", "confidence": 0.88, "reason": "..." }
    ],
    "metrics": ["Accuracy", "Precision", "Recall", "F1", "ROC-AUC"],
    "experiment_plan": {
```

```
"baseline": "Logistic Regression",
"cv": "5-fold",
"tuning": "Bayesian Optimization"
},

"responsible_ai": {
"risks": ["class imbalance", "demographic parity"],
"mitigations": ["stratified sampling", "fairness constraints"]
},

"critic": {
"attempts": 1,
"passed": true,
"feedback": "All recommendations consistent with requirements"
},

"blueprint": {
"name": "Customer Churn Prediction",
"description": "...",
"pipeline": [...],
"architecture": [...],
"timeline": [...]
},

"artifacts": {
"zip_path": "storage/exports/session-1234/project.zip",
"generated_at": "2024-01-15T10:45:00Z"
}
}
```

State Access: Frontend reads sanitized subsets via public API. n8n reads/writes via internal routes. Agents never access DB directly—they receive `state_slice` in request body and return JSON output.

8. API Contracts – Teammate Interfaces

Clear API contracts between Frontend ↔ Backend and Backend ↔ n8n eliminate integration surprises.

Frontend ↔ Backend (Public):

Method	Endpoint	Purpose
POST	/api/v1/sessions	Create session { idea } → start pipeline
GET	/api/v1/sessions/{id}	Poll status, step, questions, errors
POST	/api/v1/sessions/{id}/clarifications	Submit answers { answers[] } → resume pipeline
GET	/api/v1/sessions/{id}/blueprint	Fetch final blueprint when ready
GET	/api/v1/sessions/{id}/download	Download generated ZIP file

Backend ↔ n8n (Internal Webhooks):

Direction	Endpoint	Payload
Backend → n8n	POST /webhook/start	{ session_id, raw_idea }
Backend → n8n	POST /webhook/resume	{ session_id, clarification_answers }

n8n ↔ Backend (Internal):

Method	Endpoint	Purpose
GET	/internal/state/{session_id}	Load full WorkflowState
PATCH	/internal/state/{session_id}	Merge agent output into state
POST	/internal/codegen/generate	Blueprint JSON → starter project ZIP

n8n ↔ AI Service:

All agents follow the same request/response contract:

```
POST /agents/{agent_name}/run
```

Request:

```
{
  "session_id": "uuid",
  "state_slice": { ... }, # Relevant portion of WorkflowState
  "config": { ... } # Optional: model selection, temperature, etc.
}
```

Response:

```
{
  "success": true,
  "output": { ... }, # Agent-specific structured output
  "usage": { "tokens": 1234, "model": "gpt-4" },
  "error": null
}
```

9. RAG Implementation

RAG (Retrieval-Augmented Generation) powers agent reasoning by grounding recommendations in real documentation.

Document Sources:

Scikit-learn Documentation, PapersWithCode, Kaggle Guides, Google ML Documentation, Machine Learning Books, Research Papers

Pipeline:

1. INGEST (Offline)
 - Collect documents from sources
 - Chunk into 1000-token segments (200-token overlap)
 - Generate embeddings (Sentence Transformers)
 - Store in PostgreSQL + pgvector
2. RETRIEVE (Online, called by n8n)
 - Receive query + filters from n8n
 - Convert query to embedding
 - Similarity search in pgvector
 - Return top-k chunks with metadata
 - n8n integrates chunks into state
3. RAG Agent
 - Receives chunks via state_slice
 - LLM generates response grounded in chunks
 - Outputs: refined recommendations + citations

Why pgvector (not separate vector DB)? At project scale, pgvector performance is sufficient. One database eliminates management overhead and simplifies backups. If later you need Qdrant or Pinecone, the retriever module abstracts the switch.

10. Agent Design Principles

STATELESS DESIGN

Agents receive `state_slice` in request → process → output JSON

No agent reads directly from DB or calls another agent

n8n orchestrates all agent-to-agent communication

SINGLE RESPONSIBILITY

Each agent does ONE thing:

- Requirement: extract problem, domain, constraints
- Clarification: ask follow-up questions
- Dataset: discover & score datasets
- Model: recommend models with explanations
- Metrics: suggest evaluation metrics
- Experiment: plan baselines and tuning
- RAI: flag bias and fairness concerns
- Critic: validate all recommendations
- Blueprint: generate final project blueprint

STRUCTURED OUTPUT

Every agent returns validated JSON (Pydantic or JSON schema)

Never free-form text → always parseable and reusable

SHARED LLM CLIENT

All agents import `llm/client.py` for:

- Provider API calls with timeout
- Model selection from config
- Token usage logging
- Structured output validation
- Retry on parse failure

Each agent owns its prompt and parsing—client handles plumbing only.

CRITIC LOOP

Critic agent validates all prior recommendations

If failed: n8n retries the flagged agent (max 2 attempts)

If passed: proceed to Blueprint

NO AGENT-TO-AGENT IMPORTS

FORBIDDEN: `from agents.model import recommend_model`

ALLOWED: n8n calls model agent → merges JSON → passes to next agent

11. n8n Workflow Orchestration

n8n is the single orchestrator. It sequences agents, handles waits (for clarifications), implements retry logic, and merges outputs into shared state.

Why n8n?

- ✓ Visual Pipeline — Easy to explain in viva and demo
- ✓ Wait/Resume — Built-in for clarification loops without custom job queue
- ✓ Retry Logic — Handles flaky LLM calls elegantly
- ✓ Your Ownership — Core project contribution (viva interview will ask about this)
- ✓ No Custom Orchestration Code — Saves time vs. building Celery/RQ pipeline

Workflow Stages (Logical Order):

1. Load state from PostgreSQL
2. Requirement Agent
3. Clarification Agent → [If questions: WAIT FOR USER RESPONSE]
4. RAG Retrieve (get chunks from pgvector)
5. Dataset Agent
6. Model Agent
7. Metrics Agent
8. Experiment Agent
9. Responsible AI Agent
10. Critic Agent → [If validation fails & attempts < 2: RETRY flagged agent]
11. Blueprint Agent (generate final blueprint)
12. Code Generator (Backend endpoint)
13. Mark workflow completed & set status

Each n8n Node Pattern: GET state → extract slice via expressions → POST to agent → merge response → PATCH state

12. Deployment & Scalability

This architecture is designed to run on a single VPS or Docker Compose locally. It can scale horizontally if needed, but the current design prioritizes simplicity.

Deployment Options:

Option	Setup	Best For
Local Dev	Docker Compose	Development & Demo
VPS (Render/Railway)	Docker containers	Small production use
AWS	ECS + RDS + S3	Scaling beyond 100s of users
Kubernetes	Helm charts	Enterprise deployment (overkill for now)

Docker Compose (Local):

```
services:
  postgres:
    image: postgres:15-alpine
    environment:
      POSTGRES_PASSWORD: dev
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"

  pgvector:
    # PostgreSQL with pgvector extension
    build: ./docker/pgvector
    ports:
      - "5433:5432"

  backend:
    build: ./backend
    ports:
      - "8000:8000"
    environment:
      DATABASE_URL: postgresql://user:pass@postgres:5432/buildml
      N8N_WEBHOOK_URL: http://n8n:5678/webhook/...

  ai_service:
    build: ./ai_service
    ports:
      - "8001:8000"
    environment:
      OPENAI_API_KEY: ${OPENAI_API_KEY}
      DATABASE_URL: postgresql://user:pass@postgres:5432/buildml

n8n:
```



```
image: n8nio/n8n
ports:
- "5678:5678"
environment:
N8N_BASIC_AUTH_ACTIVE: "true"
N8N_BASIC_AUTH_USER: admin
N8N_BASIC_AUTH_PASSWORD: ${N8N_PASSWORD}
DATABASE_TYPE: postgres
DATABASE_URL: postgresql://user:pass@postgres:5432/n8n
volumes:
- n8n_data:/home/node/.n8n
- ./n8n/workflows:/home/node/.n8n/workflows:ro

volumes:
postgres_data:
n8n_data:
```

13. Viva Explanation (30 Seconds)

"BuildMyML is an AI ML consultant platform built with React frontend and FastAPI backend. When a user submits an ML idea, the backend saves the session to PostgreSQL and triggers an n8n workflow. n8n orchestrates our multi-agent pipeline—requirement analysis, clarification questions, RAG retrieval for documentation, dataset and model recommendations, evaluation metrics, experiment planning, responsible AI checks, and critic validation—by calling our AI service modules over HTTP. Agents never call each other; n8n controls execution order and handles waits for clarifications and retries for critic feedback.

RAG uses pgvector in PostgreSQL to retrieve relevant documentation chunks. Each agent processes a state slice, returns structured JSON, and n8n merges the output back into the shared WorkflowState.

The final blueprint JSON is passed to our teammate's code generator, which produces a downloadable starter project ZIP with folder structure, README, and baseline code.

We deployed this on Docker Compose with four services: Frontend, Backend, AI Service, and n8n, all backed by PostgreSQL. This design is modular, testable, and viva-friendly—one diagram, one database, one orchestrator."

14. Production-Inspired Practices

Practice	Your Implementation	Benefit
API Contracts	OpenAPI specs on Backend + AI Service	Clear interfaces; easy integration testing
Service Boundaries	Two FastAPI apps + clear module folders	Scalability; ownership clarity
Structured I/O	JSON schema per agent output	Parseable, reusable, debuggable
Observability	session_id in all logs + optional step_logs	Easier debugging; audit trail
Idempotency	X-Idempotency-Key on agent POSTs	Safe retries without duplicates
State Management	Single WorkflowState in PostgreSQL JSONB	One source of truth; no race conditions
Vector Search	pgvector instead of managed vector DB	Simpler ops; one database
Versioning	Blueprint schema versioned ("blueprint_version": "prod")	Ensures proper handoffs
Retry Logic	n8n built-in retries + explicit critic loop	Handles transient LLM failures gracefully

15. Trade-offs & Simplified Choices

Simplification	Trade-off	Mitigation
No Redis	Status reads hit PostgreSQL; slight delay	Index by session_id; poll interval 2–3s
No S3	ZIPs lost if disk wiped	Document backup path; export workflows to git
Polling vs SSE	Slightly less "live" UX	Simpler frontend; upgrade to SSE in v2
Two apps vs one	Two processes to run locally	Single docker-compose.yml abstracts this
pgvector vs Qdrant	Weaker at very large scale (100M+ vectors)	Vector KB is small/medium; sufficient now
Modular monolith	Cannot scale agents independently	Irrelevant for project scope
n8n dependency	Extra container to manage	Export workflow JSON to git; document startup
Local file storage	Not cloud-native	Teammate Export module abstracts path
Sequential agents	Slower pipeline than parallel	Parallel Dataset+Model in n8n optional later

Summary: Every trade-off is intentional and justified by project scope. You sacrificed neither modularity, testability, scalability path, nor explainability.

16. Architecture Status & Next Steps

Status: FINALIZED ■

This architecture is locked and ready for implementation. Every component has been justified. Every module has an owner. Every API contract is defined.

One Open Fork (Recommended Choice):

MONOREPO LAYOUT

Option A: Two FastAPI Entrypoints (RECOMMENDED)

```
monorepo/  
  ■■■ backend/  
  ■ ■■■ main.py # FastAPI app  
  ■■■ ai_service/  
  ■ ■■■ main.py # FastAPI app  
  ■■■ frontend/  
  ■ ■■■ src/  
  ■■■ n8n/  
  ■■■ workflows/
```

Advantage: Clear ownership boundary (you vs teammate) visible in folder structure
Perfect for viva ("Hima owns ai_service; Aayush owns backend")
No microservice deployment pain (single docker-compose.yml)

Option B: Single FastAPI App with Prefixes

```
monorepo/  
  ■■■ api/  
  ■■■ main.py # FastAPI app with /api/v1 + /internal routes  
  ■■■ public.py # Routers for /api/v1 (public)  
  ■■■ internal.py # Routers for /internal (n8n only)
```

Advantage: Single process to run

Disadvantage: Ownership boundary less clear; needs code organization discipline

Recommendation: Use Option A (two entrypoints in one monorepo). It's clearer for a viva, easier to test independently, and costs nothing extra in deployment.

17. Summary & Key Takeaways

- ✓ BuildMyML is an AI ML consultant platform—not a complex orchestration infrastructure
- ✓ Simplified architecture for 8–10 weeks + small team
- ✓ Four deployment units: Frontend, Backend, AI Service, n8n
- ✓ One database (PostgreSQL + pgvector); no Redis, S3, or separate vector DB
- ✓ Nine AI agents (all stateless; n8n orchestrates)
- ✓ One shared state (WorkflowState in JSONB)
- ✓ Clear API contracts (public + internal)
- ✓ RAG pipeline using pgvector for retrieval
- ✓ Critic loop for validation + retries
- ✓ Code generator produces starter project ZIPs

PRODUCTION-INSPIRED WITHOUT ENTERPRISE OVERHEAD

- ✓ API versioning and contracts
- ✓ Service boundaries and ownership
- ✓ Structured I/O and error handling
- ✓ Logging and observability
- ✓ State versioning and audit trail

READY FOR VIVA

- ✓ One diagram, one orchestrator, one database
- ✓ Clear team responsibilities
- ✓ Explainable design choices
- ✓ Scalability path (no regrets)

READY FOR IMPLEMENTATION

- ✓ Module ownership defined
- ✓ API contracts locked
- ✓ Technology stack finalized
- ✓ Folder structure templated
- ✓ Next step: build folder structure + skeleton code